

CSci 530: Operating Systems (xv6) Class Resources

xv6 a simple Unix-like teaching Operating System

Derek Harter

Summer 2026

Contents

1	Introduction	2
2	Getting Started	2
2.1	Testing Your Installation	2
2.2	xv6-riscv Code and Book Source Setup	2
2.3	Compiling and Running xv6-riscv Code	3
2.4	Running xv6 with gdb Debugger	4
2.5	xv6 Text Book LaTeX Source Build	5
3	Class Resources	6

1 Introduction

Materials developed for [East Texas A&M University](#) course CSci 530: Operating Systems. These materials were initially developed in the Spring 2026 and Summer 2026 semesters. This course is based on the MIT Operating System course 6.1810 Operating System Engineering. We use the xv6 Unix-like teaching operating system to present OS concepts and create assignments to build or modify portions of the working xv6 OS for the course.

- [xv6-riscv code repository](#)
- [xv6-riscv-book](#)
- [MIT 6.1810: Operating System Engineering](#)

2 Getting Started

This repository contains a Debian 13 (trixie) Docker / DevContainer setup. These instructions assume that you have Docker installed and are running this repository inside of a Docker and/or VSCode DevContainer. See the [Git / Docker / VSCode Class Development Environment Configuration](#) instructions for installing and setting up git, docker and VSCode. The following steps assume you are running inside of the container provided, and have the RISC-V cross compilation and QEMU RISC-V emulator tools installed and available.

2.1 Testing Your Installation

The RISC-V gnu gcc toolchain and QEMU (Quick EMUlator) are installed as part of the class container. Check you have the correct tools available using the following commands:

```
(base) student@xv6os:/workspaces/csci530-os-xv6$ riscv64-linux-gnu-gcc --version
riscv64-linux-gnu-gcc (Debian 14.2.0-19) 14.2.0
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

(base) student@xv6os:/workspaces/csci530-os-xv6$ qemu-system-riscv64 --version
QEMU emulator version 10.0.11 (Debian 1:10.0.11+ds-0+deb13u1)
Copyright (c) 2003-2025 Fabrice Bellard and the QEMU Project developers
```

If these tools are not available, you may not be running in the correct Docker container yet, or have not yet configured your environment correctly. These tools should be installed automatically as part of the creation of the Docker DevContainer. Do not proceed until you have all of the tools installed and available for use in your development environment.

2.2 xv6-riscv Code and Book Source Setup

This repository does not contain a copy of the xv6 teaching operating source code. Instead a clone of the official repository of the operating system source and book source code is made for use in this running container. The repository `.devcontainer/Dockerfile` should, however, setup and install all of the needed packages to support compiling and running the xv6 teaching operating system. If needed, you can clean these source directories and (re)set them up at anytime.

Run the following command to clone the source code and LaTeX book code repositories. These will be cloned into a subdirectory named `xv6`

```
(base) student@xv6os:/workspaces/csci530-os-xv6$ make setup
mkdir -p xv6
...
cp -f xv6/patch/Makefile.xv6-riscv-book xv6/xv6-riscv-book/Makefile
cp -f xv6/patch/Makefile.xv6-riscv-src-booklet xv6/xv6-riscv-book/xv6-riscv-src-booklet/Makefile
cp -f xv6/patch/runoff xv6/xv6-riscv-book/xv6-riscv-src-booklet/runoff
```

This command is safe to rerun, and will perform a `git update` if the git repositories have already been cloned. The source code for the operating system is placed in the `xv6/xv6-riscv` subdirectory. The source code to build the

companion textbook is placed in `xv6/xv6-riscv-book`. Some of the make procedure to build the book are patched to work in this environment.

You can clone the repository in a separate working directory if you desire. The main purpose of this container is to provide a working linux environment with the correct tools installed, so that you can build the xv6 source code and run it in the qemu emulator. If needed, you can run:

```
(base) student@xv6os:/workspaces/csci530-os-xv6$ make clean
rm -rf xv6/xv6-riscv xv6/xv6-riscv-book
```

To delete all of the working repository copies, and rerun setup. This might be necessary if you change some configuration in the running code that breaks your build or system, and need to restart from a clean build repository.

2.3 Compiling and Running xv6-riscv Code

You can and should be examining the code of xv6, and making modifications to the system and user programs as you read the text. The code of the teaching operating system is the primary source of study in this class. By the end of this class, you should be relatively familiar with all of the system and user space code in xv6, and have a general understanding of how the operating system works.

To build the xv6 operating system source code and start it running in the QEMU emulator, do the following.

Change into the repository for the xv6 os source code:

```
(base) student@xv6os:/workspaces/csci530-os-xv6$ cd xv6/xv6-riscv
(base) student@xv6os:/workspaces/csci530-os-xv6/xv6/xv6-riscv$
```

If the correct RISC-V toolchain are installed, the `make qemu` target will compile everything and start xv6 running in the QEMU emulator.

```
(base) student@xv6os:/workspaces/csci530-os-xv6/xv6/xv6-riscv$ make qemu
riscv64-linux-gnu-gcc -march=rv64gc -g -c -o kernel/entry.o kernel/entry.S
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer
-ggdb -gdwarf-2 -march=rv64gc -std=gnu99 -MD -mmodel=medany -ffreestanding
-fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen
-fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log
-fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc
-fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf
-fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie
-c -o kernel/start.o kernel/start.c

...

qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3
-nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ <C-a h>
C-a h    print this help
C-a x    exit emulator
C-a s    save disk data back to file (if -snapshot)
C-a t    toggle console timestamps
C-a b    send break (magic sysrq)
C-a c    switch between console and monitor
C-a C-a  sends C-a
```

```
<C-a x>
QEMU: Terminated
```

A lot of the output from the compilation has been omitted in the ... above, though usually the compilation of all of the code should complete quite quickly. Notice that we left in the invocation of the `qemu-system-riscv64` command that actually starts up the QEMU emulation of a RISC-V processor running the xv6 kernel. Some of the options here specify which kernel image to use, the amount of memory available, and device drivers and file system images to use. The `hart 2 starting` and `hart 1 starting` refer to the emulated “hardware threads”. The default is for 2 hardware threads to started and available in the emulation. A hardware thread is a RISC-V terms for a CPU processor core, so we are emulating 2 CPU cores by default here. (Note: the `-smp 3` flag controls the number of symmetric multi-processors emulated by QEMU so you may ask why there are only two hardware threads when `-smp` is set to 3. We may talk about this later in the class.)

To get access to the QEMU escape sequences use `<C-a h>` as shown (hold the control key and hit `a`, followed by `h`). You can cause the QEMU emulator to exit using the `<C-a x>` sequence.

If needed, you can first perform a `make clean` before invoking `make qemu`. This will ensure that all files are rebuilt for the operating system kernel and user images.

2.4 Running xv6 with gdb Debugger

To step through xv6 code using gdb, in your first terminal use the following command to start xv6 as a gdb service that can be connected to remotely. We are essentially invoking the `qemu-gdb` target here, to start the emulator in gdb mode. The `CPUS=1` flag limits the emulator to a single cpu, which usually makes debugging easier:

```
(base) student@xv6os:/workspaces/csci530-os-xv6/xv6/xv6-riscv$ make CPUS=1 qemu-gdb
sed "s/:1234/:26000/" < .gdbinit.tmpl-riscv > .gdbinit
*** Now run 'gdb' in another window.
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M
-smp 1 -nographic -global virtio-mmio.force-legacy=false
-drive file=fs.img,if=none,format=raw,id=x0
-device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0 -S -gdb tcp::26000
```

You will notice that you don’t get a prompt like before. Also if you look closely, the `-smp 1` flag has been set to use a single cpu, and the `-S -gdb tcp:26000` flags have been added, which runs the emulator in gdb debugging mode that can be attached to at port 26000 on the local host.

Open another terminal, and perform the following. You need to open the terminal inside of your Docker/DevContainer to access the gdb port that was opened.

A file named `.gdbinit` should have been created in the `xv6-riscv` source code directory, as shown in the output above. The file has the following contents:

```
(base) student@xv6os:/workspaces/csci530-os-xv6$ cd xv6/xv6-riscv
(base) student@xv6os:/workspaces/csci530-os-xv6/xv6/xv6-riscv$ cat .gdbinit
set confirm off
set architecture riscv:rv64
target remote 127.0.0.1:26000
symbol-file kernel/kernel
set disassemble-next-line auto
set riscv use-compressed-breakpoints yes
```

You may get a warning if you don’t specify a `safe-path` for gdb. So before running gdb, perform the following to create a global `gdbinit` configuration file that will always allow you to connect a gdb session. The following creates a file named `~/.config/gdb/gdbinit`. You could instead create and add in this line using your VSCode text editor as well. Once created you won’t have to recreate this file, so the following only needs to be done 1 time:

```
(base) student@xv6os:/workspaces/csci530-os-xv6/xv6/xv6-riscv$
mkdir -p ~/.config/gdb; echo "set auto-load safe-path /" >> ~/.config/gdb/gdbinit
```

Now from the same directory where the `.gdbinit` file was generated, run this command to start a gdb debugger. This connects to the open port specified in the `.gdbinit` file found in your current directory:

```
(base) student@xv6os:/workspaces/csci530-os-xv6/xv6/xv6-riscv$ gdb-multiarch -tui
```

Some basic gdb commands:

Command	What it does
<code>b param / break param</code>	Adds a breakpoint on symbol <code>param</code>
<code>c / continue</code>	Will continue execution until next breakpoint
<code>s / step</code>	Step into the current line, entering function call if needed
<code>n / next</code>	Continue to next line, not entering function call on current line
<code>watch expression</code>	Will stop (break) when value of expression changes
<code>rwatch expression</code>	Will stop (break) when value of expression is read
<code>q / quit</code>	Quit from the debugging session

I usually set a break on the `_entry` and/or on the `start` symbols / functions then do a `continue` to jump to that point and step from there.

```
GNU gdb (Debian 16.3-1) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
...
determining executable automatically. Try using the "file" command.
0x00000000000001000 in ?? ()
(gdb) b _entry
Breakpoint 1 at 0x80000000: file kernel/entry.S, line 12.
(gdb) b start
Breakpoint 2 at 0x800000058: file kernel/start.c, line 16.
(gdb) c
Continuing.

Breakpoint 1, _entry () at kernel/entry.S:12
12          la sp, stack0
```

2.5 xv6 Text Book LaTeX Source Build

The `LATEX`source to build the open source textbook and companion source code booklet is also cloned as part of the setup process. A copy of the text book is available in the `~/docs/xv6-book.pdf` directory of this repository, which includes the instructors highlights and notes in the text. If you want your own copy of the pdf book or booklet, to make your own notes, do the following:

```
(base) student@xv6os:/workspaces/csci530-os-xv6$ cd xv6/xv6-riscv-book/
(base) student@xv6os:/workspaces/csci530-os-xv6/xv6/xv6-riscv-book$ make

... a lot of output from the LaTeX build

(base) student@xv6os:/workspaces/csci530-os-xv6/xv6/xv6-riscv-book$ ls *.pdf
book.pdf  xv6-src-booklet.pdf
```

The result should be the `book.pdf` file, which is the companion open source text book for the class code, and the `xv6-src-booklet.pdf`, which is a numbered source code listing of the actual xv6 os code.

Note: You will need to install additional packages for the `LATEX`build. See the `.devcontainer/Dockerfile` to uncomment out the needed package install command, or try running the following to install them, this install takes quite a bit of time to complete:

```
$ sudo apt-get install -y snakemake pandoc python3-pweave texlive  
texlive-fonts-recommended texlive-latex-recommended biber  
texlive-bibtex-extra texlive-fonts-extra texlive-science  
ghostscript gnuplot locales psutils a2ps
```

Note: I don't personally find the source code booklet too useful, as it is easier in this environment to simply open up the source file in the VSCode editor when reading the text book. Also the booklet build is currently a bit broken, and page breaks are not quite coming out correctly in the booklet, which needs to be fixed at some point.

3 Class Resources

- [Setting Up Git / Docker / VSCode DevContainers](#)
- [GDB Cheat Sheet](#)
- [xv6-riscv code repository](#)
- [xv6-riscv-book](#)
- [MIT 6.1810: Operating System Engineering](#)